

Introduction to API Security

An API exposes data to any client that can send an HTTP request, so it cannot rely on a user interface to control access. Our API serves mobile money transactions: names, phone numbers, amounts and balances. If it were left open, anyone could read, change or delete people's financial records.

API security comes down to a few core protections:

Authentication: confirming who is making the request.

Authorisation: deciding what that user is allowed to do.

Input validation: rejecting malformed or malicious data before it reaches storage.

Encryption in transit: using HTTPS so requests and credentials cannot be read on the network.

Rate limiting: stopping brute-force attempts and abuse.

Our API applies authentication and validation. Every endpoint requires HTTP Basic Authentication, checked before routing, so an unauthenticated request never reaches the data and receives 401 Unauthorized. Credentials come from environment variables and are compared in constant time. POST and PUT bodies are validated for required fields, types and allowed values, and ids are always assigned by the server. HTTPS, per-user roles and rate limiting are not implemented; these gaps are discussed in the Basic Auth reflection later in this report.

API Endpoint Documentation

Base URL: <http://localhost:8000>. Every request needs an Authorization: Basic header (curl -u username:password). All responses are JSON. Full documentation is in docs/api_docs.md.

Method	Endpoint	Description	Success
GET	/transactions	List all transactions	200
GET	/transactions/{id}	Get one transaction	200
POST	/transactions	Create a transaction	201
PUT	/transactions/{id}	Partially update a transaction	200
DELETE	/transactions/{id}	Delete a transaction	200

GET /transactions

Request

```
curl -u admin:password123 http://localhost:8000/transactions
```

Response: 200 OK

```
[
  {"id": 1, "type": "received", "amount": 2000.0, "sender": "Jane Smith", ...},
  ...
]
```

Errors: 401 Unauthorized.

GET /transactions/{id}

Request

```
curl -u admin:password123 http://localhost:8000/transactions/1
```

Response: 200 OK

```
{"id": 1, "type": "received", "amount": 2000.0, "sender": "Jane Smith",
"receiver": null, "transaction_id": "76662021700", "fee": null,
"balance": null, "timestamp": "10 May 2024 4:30:58 PM", "raw_body": "...}"
```

Errors: 401 Unauthorized; 404 {"error": "transaction 9999 not found"}.

POST /transactions

Required: type, amount, timestamp. Optional fields default to null. The server assigns the id.

Request

```
curl -u admin:password123 -X POST http://localhost:8000/transactions \
-H "Content-Type: application/json" \
-d '{"type": "payment", "amount": 5000, "timestamp": "2026-09-29 10:00:00"}'
```

Response: 201 Created

```
{"type": "payment", "amount": 5000, "timestamp": "2026-09-29 10:00:00",
"id": 1692, "sender": null, "receiver": null, "transaction_id": null,
"fee": null, "balance": null, "raw_body": null}
```

Errors: 400 with all problems listed, e.g. {"errors": ["missing required field: type", "amount must not be negative"]}; 401 Unauthorized.

PUT /transactions/{id}

Partial update: send only the fields to change. The id cannot be changed.

Request

```
curl -u admin:password123 -X PUT http://localhost:8000/transactions/1692 \
-H "Content-Type: application/json" -d '{"amount": 7500}'
```

Response: 200 OK

```
{"type": "payment", "amount": 7500, "timestamp": "2026-09-29 10:00:00", "id": 1692, ...}
```

Errors: 400 for an empty body, invalid values or unknown fields such as id; 401 Unauthorized; 404 not found.

DELETE /transactions/{id}

Request

```
curl -u admin:password123 -X DELETE http://localhost:8000/transactions/1692
```

Response: 200 OK

```
{"deleted": 1692}
```

Errors: 401 Unauthorized; 404 not found.

Error codes

Code	Meaning	Response body
400	Invalid request body	{"errors": [...]}
401	Missing or wrong credentials	{"error": "Unauthorized"}
404	Transaction or route not found	{"error": "..."}
501	Unsupported method (e.g. PATCH)	HTML error page

Results of DSA Comparison

So the task here was simple on paper. Take our 1691 transactions, find one by its id, two different ways, and see which one actually wins. Turns out the answer isnt surprising, but the numbers still tell a good story.

Method 1: Linear Search

This one just walks through the list, one record at a time, checking the id as it goes. Its the obvious way to do it if you dont think too hard. Works fine on small data. Problem starts when the list gets long and the thing you want is near the end.

Method 2: Dictionary Lookup

Before searching, we build a dictionary once, id maps straight to the record. After that a lookup is basically instant, python hashes the key and jumps right to it. No scanning, no waiting.

The Test

We timed both methods on the real dataset, all 1691 parsed transactions, not some small sample. Picked six ids spread across the list, from the very first record to near the last one, and ran each search 2000 times per id so the numbers wouldn't be random noise. Here is what came back.

Record ID	Linear search (ms)	Dictionary lookup (ms)	Dictionary speedup
1	0.427	0.314	1.4x
100	9.527	0.319	29.8x
400	37.037	0.352	105.1x
800	72.126	0.315	229.1x
1200	115.123	0.326	353.1x
1690	160.870	0.322	500.2x

Notice the dictionary column. It barely moves, stays around 0.3 ms the whole way through, doesn't matter if the record is first or last. Linear search on the other hand just keeps climbing, by the time we hit record 1690 it's taking 500 times longer than the dictionary. That's not a small difference anymore, that's the whole point of this exercise.

Reflection

Why is dictionary lookup faster, really? Linear search is $O(n)$, it has to check every record one by one until it either finds the match or runs out of list. So worst case, time grows with how many records you have and where the target sits. A dictionary is $O(1)$, it hashes the key and jumps straight to the slot where the value lives. Doesn't matter if there are 10 records or 10 million, the lookup cost stays about the same.

For our api, this matters a lot, since `GET /transactions/{id}` is probably the endpoint that's going to get hit the most. Using a dict here instead of looping through a list was the obvious choice once you see these numbers.

One thing worth adding, a dictionary is great for exact id lookups but it's useless for range queries, like give me all transactions between two dates, or all transactions above 10000 RWF. For that kind of search a sorted structure would help more, something like a balanced binary search tree, or even just a sorted list with binary search. Those won't beat a dict on a single id lookup but they open up range searches that a plain hash table just can't do.

API Implementation: Building and Testing the CRUD Server

Kevin's parser provided us with `dsa/parsed_transactions.json` (1691 transactions) and Emmanuel had a script (`auth.py`, `validation.py`) ready with an explanation of how they fit into a server. The most basic of all the things – the server itself, `api/server.py` – did not exist.

There was no framework, no decorators, no json parsing or even path matching (like `/transactions/{id}`) – it was all done by hand for the simple http server.

Auth and validation

Each handler starts by looking at auth, before reading body:

```
def _require_auth(self):
    if not check_auth(self.headers):
        send_unauthorized(self)
        return False
    return True
```

Validation is second, only after auth on POST/PUT an unauthenticated request can't get that far as to have to have it validated by the JSON.

The DSA connection

Searching by dictionary was relatively constant around 0.3ms which is not bad, while searching by linear search increased by over 500x, reaching the end of the list. This has direct impact on the design of the look up:

```
LOOKUP = {t["id"]: t for t in TRANSACTIONS} # O(1), not a loop
```

GET /transactions/{id} is the endpoint most likely to get hit repeatedly, so this wasn't a style choice — it's Kevin's numbers applied to the real API.

Endpoints

Method	Path	Behavior
GET	/transactions	Full list
GET	/transactions/{id}	Dict lookup, 404 if missing
POST	/transactions	Validates, assigns next id, appends
PUT	/transactions/{id}	Partial update, validates fields sent
DELETE	/transactions/{id}	Removes record, 404 if missing

You can never use the id that is sent from the client (always set by the server).

Testing

Emmanuel's tests/test_api.sh tests all of the 10 cases required. The following three problems are all non-server.py bugs, when trying to get it up and running:

- CRLF line endings** in the script broke bash fixed by converting to LF.
- localhost resolving to IPv6** meant requests never reached the server fixed by using 127.0.0.1 directly.
- WSL vs Git Bash** typing bash on this machine launches WSL, which has a separate network stack from Windows, so it couldn't reach a server bound on Windows. Running the script through Git Bash directly fixed it.

Results

All 10 tests pass:

Test	Expected	Result
GET list, valid creds	200	200
GET list, wrong creds	401	401
GET list, no creds	401	401
GET by id	200	200
POST new record	201	201
PUT update	200	200
GET confirms PUT	200	200
DELETE	200/204	200
GET after delete	404	404
POST invalid body	400	400

Individual requests are provided in screenshots.

Screenshot 1: GET /transactions/1 with valid credentials returns 200 OK and the record.

```
10 DATA FILE = os.path.join(os.path.dirname( file ), "..", "dsa", "parsed_transactions.json")

Problems Output Debug Console Terminal Ports Postman Console

PS C:\Users\Amari\restapi_proj> curl.exe -i -u admin:password123 http://localhost:8000/transactions/1

HTTP/1.0 200 OK
Server: MoMoAPI/1.0 Python/3.13.1
Date: Tue, 29 Sep 2026 07:24:52 GMT
Content-Type: application/json
Content-Length: 406

{"id": 1, "type": "received", "amount": 2000.0, "sender": "Jane Smith", "receiver": null, "transaction_id": "76662021700", "fee": null, "balance": null, "timestamp": "10 May 2024 4:30:58 PM", "raw_body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."}
PS C:\Users\Amari\restapi_proj>
```

Screenshot 2: Wrong password returns 401 Unauthorized with the WWW-Authenticate header.

```
Problems Output Debug Console Terminal Ports Postman Console

PS C:\Users\Amari\restapi_proj> curl.exe -i -u admin:wrongpass http://localhost:8000/transactions

HTTP/1.0 401 Unauthorized
Server: MoMoAPI/1.0 Python/3.13.1
Date: Tue, 29 Sep 2026 07:28:24 GMT
WWW-Authenticate: Basic realm="MoMo API"
Content-Type: application/json
Content-Length: 25

{"error": "Unauthorized"}
PS C:\Users\Amari\restapi_proj>
```

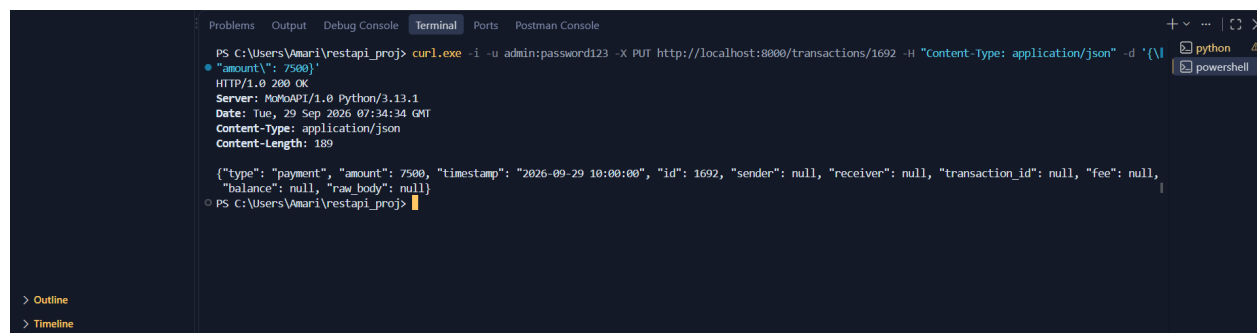
Screenshot 3: POST /transactions creates a new record, 201 Created, id 1692.



```
PS C:\Users\Amari\restapi_proj> curl.exe -i -u admin:password123 -X POST http://localhost:8000/transactions -H "Content-Type: application/json" -d '{"type": "payment", "amount": 5000, "timestamp": "2026-09-29 10:00:00"}'
HTTP/1.0 201 Created
Server: MoMoAPI/1.0 Python/3.13.1
Date: Tue, 29 Sep 2026 07:29:58 GMT
Content-Type: application/json
Content-Length: 189

{"type": "payment", "amount": 5000, "timestamp": "2026-09-29 10:00:00", "id": 1692, "sender": null, "receiver": null, "transaction_id": null, "fee": null, "balance": null, "raw_body": null}
PS C:\Users\Amari\restapi_proj>
```

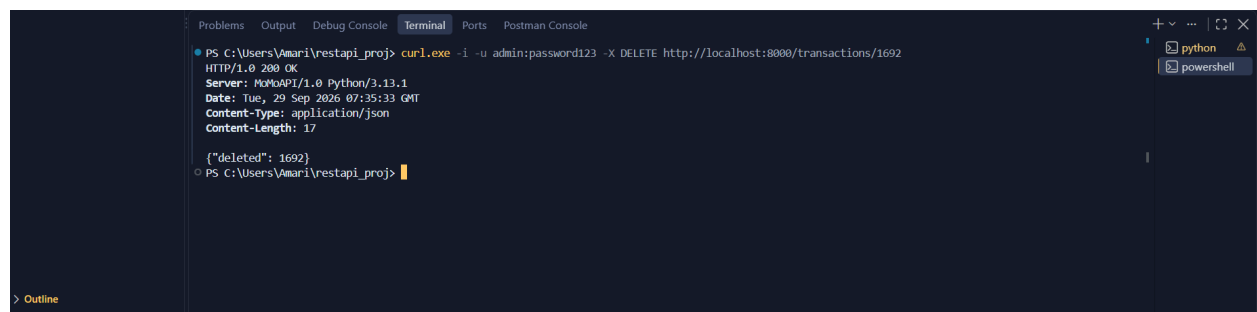
Screenshot 4: PUT /transactions/1692 updates the amount from 5000 to 7500, 200 OK.



```
PS C:\Users\Amari\restapi_proj> curl.exe -i -u admin:password123 -X PUT http://localhost:8000/transactions/1692 -H "Content-Type: application/json" -d '{"amount": 7500}'
HTTP/1.0 200 OK
Server: MoMoAPI/1.0 Python/3.13.1
Date: Tue, 29 Sep 2026 07:34:34 GMT
Content-Type: application/json
Content-Length: 189

{"type": "payment", "amount": 7500, "timestamp": "2026-09-29 10:00:00", "id": 1692, "sender": null, "receiver": null, "transaction_id": null, "fee": null, "balance": null, "raw_body": null}
PS C:\Users\Amari\restapi_proj>
```

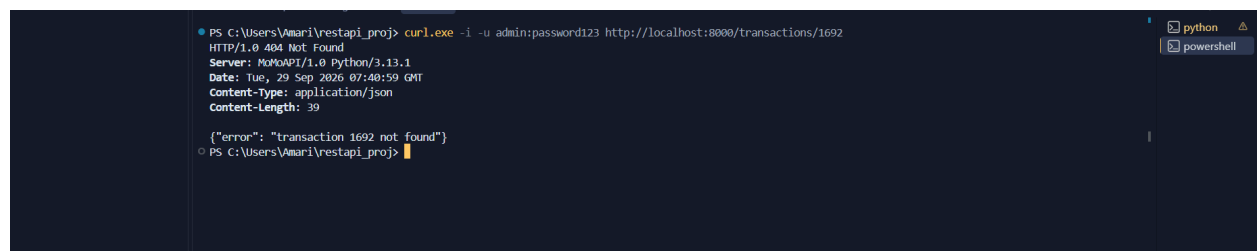
Screenshot 5: DELETE /transactions/1692 returns 200 OK and {"deleted": 1692}.



```
PS C:\Users\Amari\restapi_proj> curl.exe -i -u admin:password123 -X DELETE http://localhost:8000/transactions/1692
HTTP/1.0 200 OK
Server: MoMoAPI/1.0 Python/3.13.1
Date: Tue, 29 Sep 2026 07:35:33 GMT
Content-Type: application/json
Content-Length: 17

{"deleted": 1692}
PS C:\Users\Amari\restapi_proj>
```

Screenshot 6: GET /transactions/1692 after the delete returns 404, proving the record is gone.



```
PS C:\Users\Amari\restapi_proj> curl.exe -i -u admin:password123 http://localhost:8000/transactions/1692
HTTP/1.0 404 Not Found
Server: MoMoAPI/1.0 Python/3.13.1
Date: Tue, 29 Sep 2026 07:40:59 GMT
Content-Type: application/json
Content-Length: 39

{"error": "transaction 1692 not found"}
PS C:\Users\Amari\restapi_proj>
```

Authentication, Validation and Testing

I built the security layer: `api/auth.py` (Basic Auth), `api/validation.py` (input checks) and `tests/test_api.sh` (automated tests). Every handler in `server.py` calls auth first, then validation, before touching any data.

Authentication

`check_auth` confirms the header uses the Basic scheme, base64-decodes it (rejecting invalid base64) and splits on the first colon only, so passwords can contain colons. Credentials come from the `API_USERNAME` and `API_PASSWORD` environment variables, with dev defaults for local testing. Both values are compared with `hmac.compare_digest`, a constant-time comparison, and both are always checked, so response time never reveals whether the username alone was correct:

```
user_ok = hmac.compare_digest(username.encode("utf-8"), expected_user)
pass_ok = hmac.compare_digest(password.encode("utf-8"), expected_pass)
return user_ok and pass_ok
```

Any failure calls `send_unauthorized`, which returns 401 with a `WWW-Authenticate: Basic realm="MoMo API"` header and `{"error": "Unauthorized"}` (screenshot 2).

Validation

`validate_transaction` returns (ok, errors) and collects every problem at once rather than stopping at the first. POST requires type, amount and timestamp; PUT uses `partial=True` and checks only the fields sent (screenshot 4). It rejects unknown fields (including id, which only the server assigns), types outside the allowed list, and negative or non-numeric amounts, fees and balances. Two edge cases needed explicit handling, because Python treats `True` as a number and `json.loads` accepts NaN and Infinity:

```
if isinstance(value, bool) or not isinstance(value, (int, float)):
    return False
return math.isfinite(value)
```

Invalid input gets a 400 with the full error list. Auth runs first, so unauthenticated users never see the validation rules.

Testing

`test_api.sh` runs all 10 cases in the results table, and I tested each request manually with curl (screenshots 1 to 6). Two issues I fixed along the way: in PowerShell, curl is an alias for `Invoke-WebRequest`, so the real `curl.exe` is needed; and my first PUT and DELETE returned 404 because I used the placeholder `NEW_ID` instead of the real id, 1692. Screenshot 6 confirms the delete worked, since the record now returns 404.

Why Basic Auth is weak

Not encrypted. Base64 is encoding, so anyone who captures the header can decode the password instantly, especially over plain HTTP.

Sent on every request. More chances for the password to leak through logs or proxies.

No expiry or logout. Stolen credentials work until the password is changed.

No roles or rate limiting. One shared account can read and delete everything, and nothing stops password guessing.

Stronger alternatives

JWT: the user logs in once and gets a signed, short-lived token instead of sending the password each time. Tokens can carry roles, e.g. only admins can delete.

OAuth2: apps receive scoped, revocable access tokens and never see the password. Best for mobile apps and third-party access.

For a real MoMo API, I would use OAuth2 or JWT over HTTPS with rate limiting on login.